

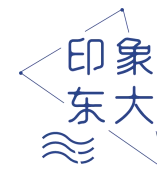


基于智慧水务平台的水质监测数据分析

汇报人：

2024. 7. 19





目录 Contents

1 项目背景

2 项目架构

3 开发流程

4 总结



1 *Part One*

项目背景



1923

► 1. 项目背景

定义：智慧水务系统是一种基于物联网、大数据等技术的智能化水务管理系统，旨在实现水资源的合理配置、高效管理和智能服务。

功能：监测与控制、数据分析与挖掘、智能决策支持、预警与应急处理、优化调度与节能降耗等。

建设目标：实现水资源的合理配置、提高供水效率、降低漏损率、提升水质监测能力。

建设原则：统一规划、分步实施、注重实效、安全可靠、经济合理

技术原理

物联网技术：实现水务设施的远程监控和数据采集。

云计算技术：实现水务数据的存储和计算资源的共享。

大数据分析：对海量数据进行处理、分析和挖掘，提供决策支持。

发展历程

初级阶段：以人工操作为主，缺乏智能化技术支持。

发展阶段：引入计算机技术，实现数据采集和监控。

成熟阶段：利用物联网、大数据等技术，提升智能化水平。

未来趋势：边缘计算等技术的融合应用，进一步提高智能化程度。

应用场景

城市供水系统：
监测水质、控制
水量、提高供水
效率

污水处理厂：自
动化处理污水、
降低能耗、提高
处理效率

农业灌溉：智能
控制水源、精准
灌溉、节约水资
源

防洪抗旱：实时
监测雨情、预测
洪水、减轻灾害
损失

智慧水务系统的优势

实时监测：对水质、水量进行实时监测，确保供水安全。

数据分析：通过数据分析，优化水资源配置，提高用水效率。

预测预警：利用大数据技术，分析水务状况，及时预警。

智能化管理：实现智能化、自动化管理，降低运营成本。

2

Part Two

项目架构



架构设计

架构概述：智慧水务系统由感知层、传输层、数据层和应用层组成。

感知层：负责水质、水位等数据的采集和传输。

传输层：通过无线网络或有线网络将数据传输至数据中心。

数据层：存储、处理和分析数据，为应用层提供支持。

应用层：根据实际需求，开发各种水务应用功能，如水质监测、远程控制等。

关键技术

物联网技术：实现
水务设施的远程监
控和数据采集。

大数据分析：对海
量数据进行分析，
提供决策支持。

云计算技术：实现
数据处理和存储的
云端化。

实施步骤：

需求分析：明确系统的功能需求和业务目标

系统设计：根据需求分析结果，设计系统架构和模块

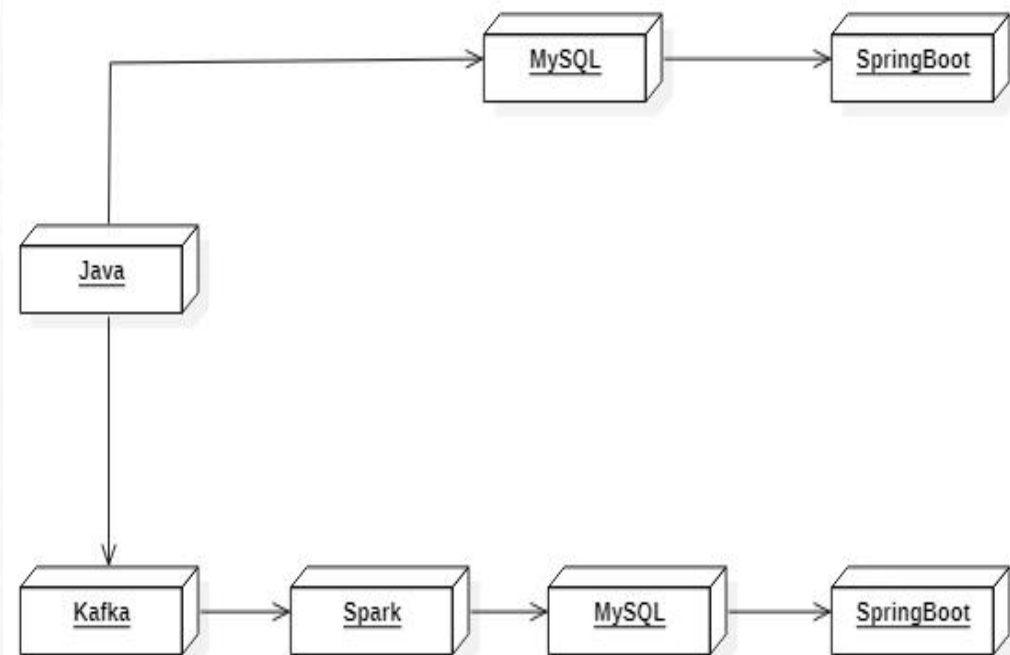
开发与集成：按照系统设计进行开发，并集成相关设备和软件

测试与调试：对系统进行测试和调试，确保系统正常运行

部署与上线：将系统部署到实际运行环境中，并进行上线操作

维护与升级：对系统进行日常维护和升级，保证系统的稳定性和安全性

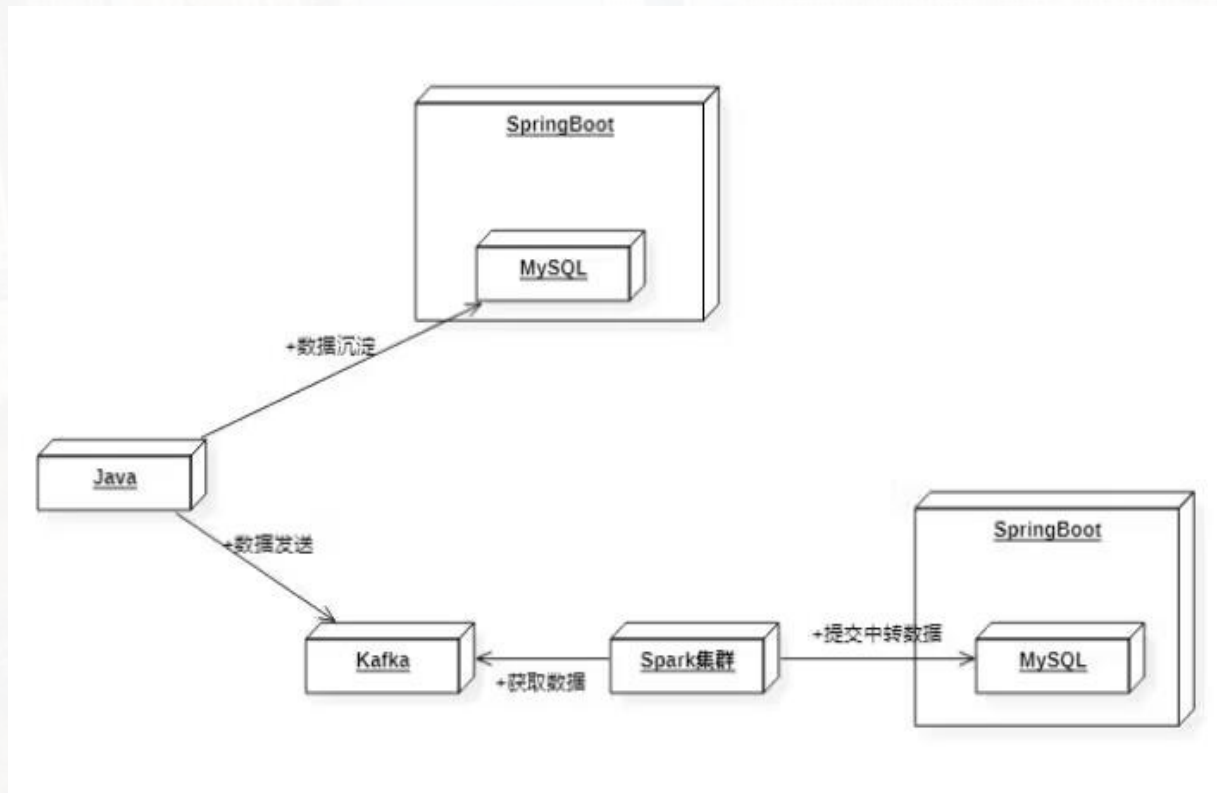
2. 项目架构



项目初步架构中，使用Java作为核心编程语言，使用SpringBoot框架开发应用程序，MySQL用于数据存储，Kafka作为数据流处理系统，Spark集群用于大规模数据处理和分析。

图2.1 项目初步整体架构图

2. 项目架构



项目实际架构中，Java程序负责从外部数据源获取数据，并通过Kafka进行消息传输。Kafka将数据传输到Spark集群，进行数据处理和分析。处理后的数据存储在MySQL数据库中，SpringBoot应用程序通过连接MySQL提供数据的读写服务。SpringBoot应用程序将处理过的数据提交到其他系统，或接收外部数据进行进一步处理。

图2.2 项目实际架构图

2. 项目架构

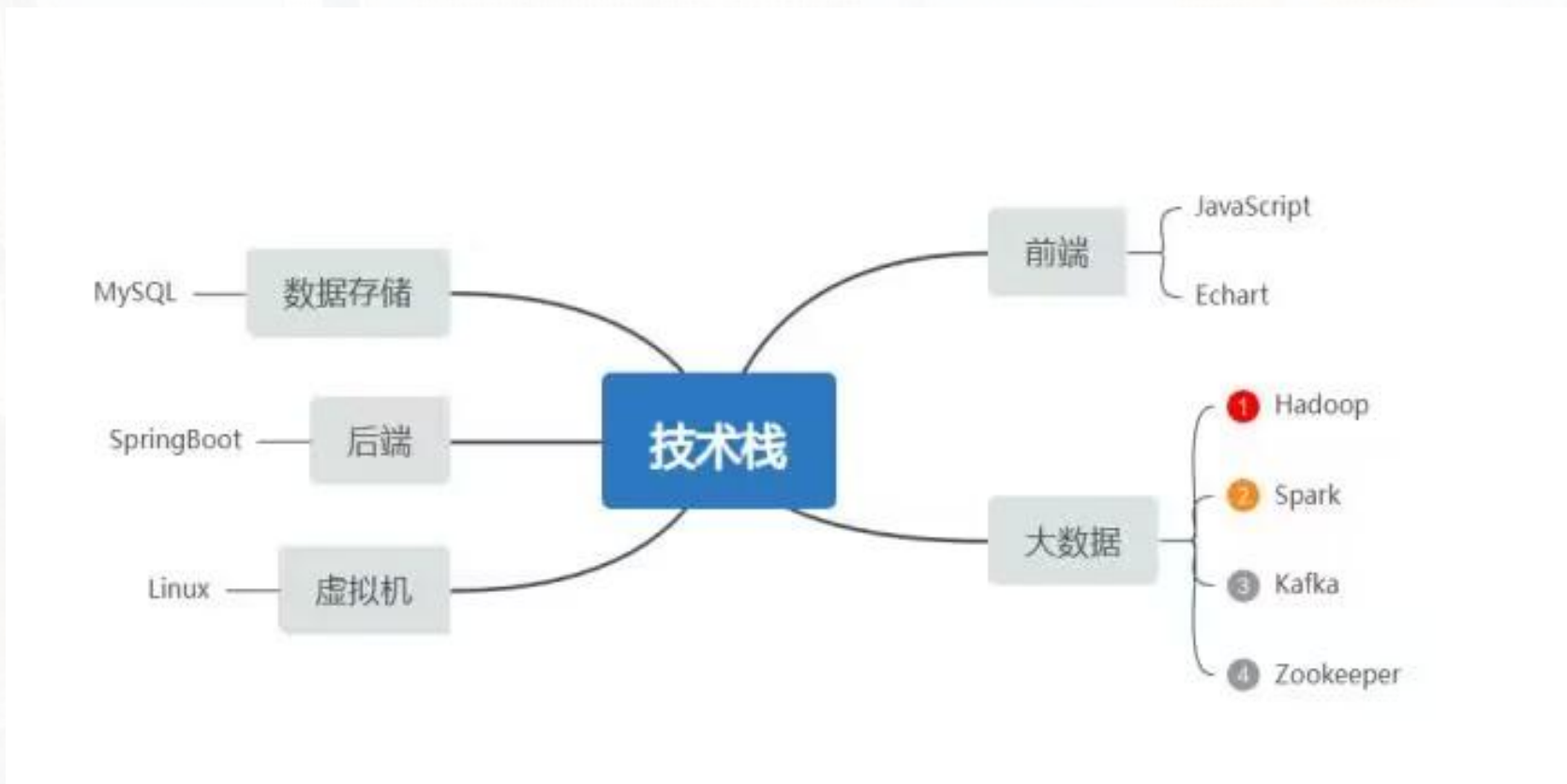


图2.3 项目技术栈

3

Part Three

开发流程



3. 开发流程

```
11 public static void main(String[] args) {
12     try {
13         //8个水厂 6条管线 0-13
14         //每个水厂/管线 5个传感器 0-4
15         int[] sensors = {1, 2, 3, 4, 5};
16
17         DatagramSocket socket = new DatagramSocket();
18         InetAddress address = InetAddress.getByName("192.168.137.2");
19         while (true) {
20             int waterworksId = Integer.parseInt(InitVar.locationList.get((int) (Math.random() * InitVar.locationList.size()))); //所属水厂编号
21             int equipmentId = sensors[(int) (Math.random() * 5)]; //设备编号
22             String equipmentType = "fs"; //设备类型:流量传感器
23
24             //流量(流速) 范围为100 - 3000, 实际范围假设 0 - 5000
25             double flowSpeed = (int) ((Math.random() * 5000) * 100000) / 100000.0;
26
27             //获取当前时间:相对于1970年1月1日的毫秒数, 后续需要转变成年月日
28             String curTime = String.valueOf(System.currentTimeMillis());
29
30             long longRTime = Long.parseLong(curTime);
31             Date dateRTime = new Date(longRTime);
32             SimpleDateFormat simpleDateFormat = new SimpleDateFormat( pattern: "yyyy-MM-dd HH:mm:ss");
33             String rTime = simpleDateFormat.format(dateRTime);
34
35             String message = waterworksId + "," + equipmentType + "," + equipmentId + "," + rTime + "," + flowSpeed;
36             byte[] data = message.getBytes();
37             socket.send(new DatagramPacket(data, data.length, address, port: 12345));
38             Thread.sleep( millis: 20000);
39         }
40     } catch (Exception e) {
```

图3.1 模拟传感器发送数据

▶ 3. 开发流程

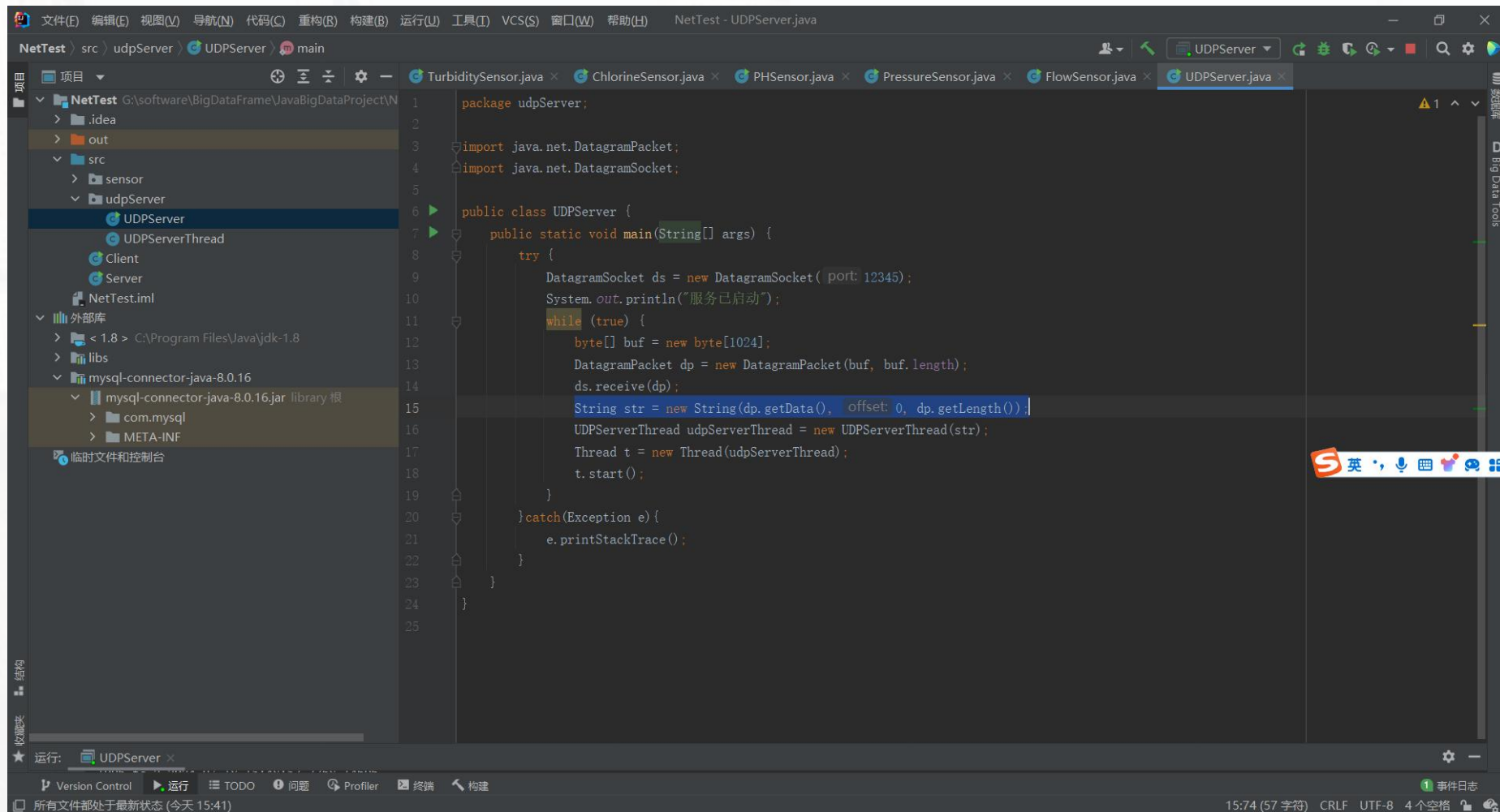


图3.2 UDP协议接收数据

UDP协议

无连接：UDP是无连接的协议，在传输数据前不需要建立连接，数据可以直接发送。这使得UDP传输数据的延迟较低。

不可靠传输：UDP不保证数据包的到达和顺序。数据包可能会丢失、重复或乱序传输。可靠性需要由应用层来处理。

数据报文：UDP将数据分成独立的报文，每个报文包含完整的地址信息，可以独立传输。

轻量级：UDP头部开销小，仅8个字节，包括源端口、目标端口、长度和校验和字段。

3. 开发流程

```
NetTest - UDPServerThread.java

Sensor.java x ChlorineSensor.java x PHSensor.java x PressureSensor.java x FlowSensor.java x UDPServer.java x UDPServerThread.java x

/* ----- 构造消息，配置语句 ----- */

switch (sensorType) {
    case "ps":
        sql = "insert into water_pressure_sensor_data(location,sensor_type,equipment_id,send_time,value)" +
            "values(" + location + "," + sensorType + "," + equipmentID + "," + sendTime + "," + value + ")";
        topic = "sensor_pressure_data";
        break;
    case "fs":
        sql = "insert into water_flow_sensor_data(location,sensor_type,equipment_id,send_time,value)" +
            "values(" + location + "," + sensorType + "," + equipmentID + "," + sendTime + "," + value + ")";
        topic = "sensor_flow_data";
        break;
    case "cs":
        sql = "insert into water_chlorine_sensor_data(location,sensor_type,equipment_id,send_time,value)" +
            "values(" + location + "," + sensorType + "," + equipmentID + "," + sendTime + "," + value + ")";
        topic = "sensor_chlorine_data";
        break;
    case "ts":
        sql = "insert into water_turbidity_sensor_data(location,sensor_type,equipment_id,send_time,value)" +
            "values(" + location + "," + sensorType + "," + equipmentID + "," + sendTime + "," + value + ")";
        topic = "sensor_turbidity_data";
        break;
    case "phs":
        sql = "insert into water_ph_sensor_data(location,sensor_type,equipment_id,send_time,value)" +
            "values(" + location + "," + sensorType + "," + equipmentID + "," + sendTime + "," + value + ")";
        topic = "sensor_ph_data";
        break;
    default:
        sql = null; break;
}
```

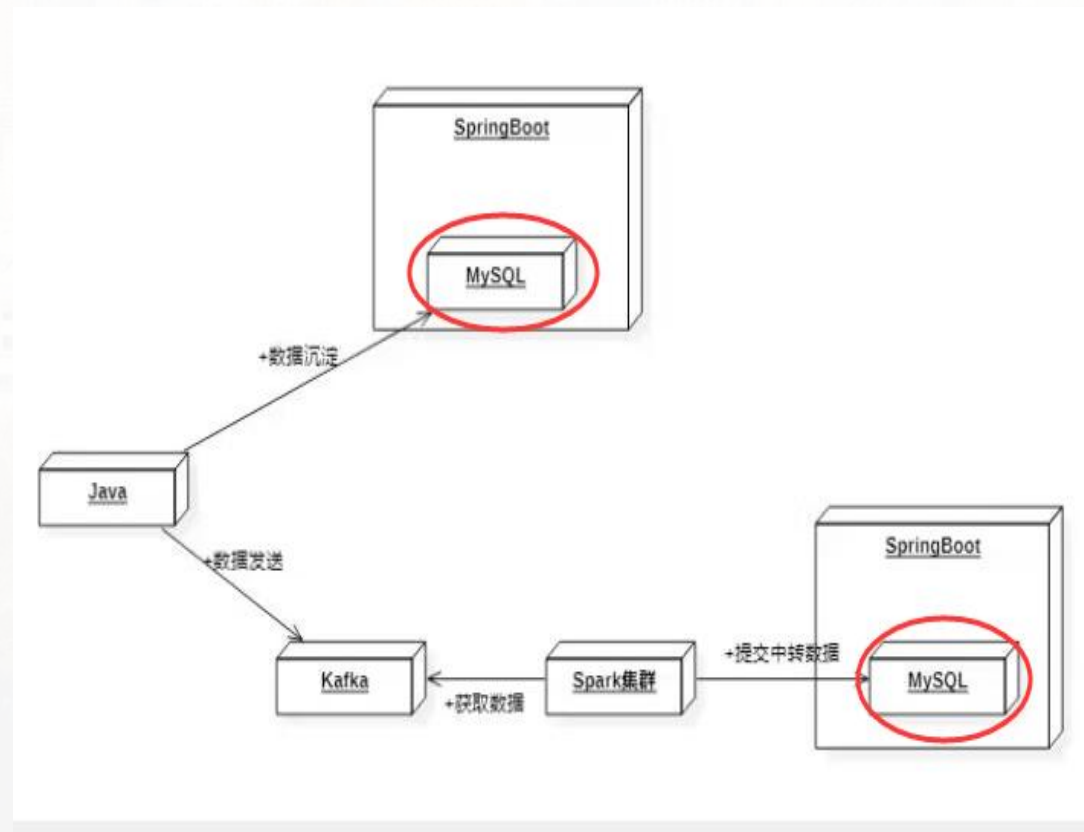


图3.3 获取传感器模拟生成的数据放入沉淀池

3. 开发流程

```
/* -----配置下游的Kafka服务器 -----*/  
Properties props = new Properties();  
//Kafka生产者  
props.put("bootstrap.servers", "192.168.56.154:9092");  
props.put("acks", "0");  
props.put("retries", 0);  
props.put("batch.size", 16384);  
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
  
//构造生产者  
Producer<String, String> producer = new KafkaProducer<>(props);
```

```
/* —— 数据传输 ——*/  
statement.executeUpdate(sql);  
  
ProducerRecord<String, String> record = new ProducerRecord<>(topic, str);  
producer.send(record);
```

图3.4 传感器数据被同时分发给Kafka消息队列

3. 开发流程

WaterMonitor

admin 在线 注销

首页

系统管理

系统监控

系统工具

基础数据

数据管理

PH值数据管理

浊度数据管理

余氯数据管理

流量数据管理

水压数据管理

PH值数据管理

搜索 重置

+ 添加 修改 删除 导出

☐	传感器位置	传感器类型	传感器编号	发送时间	数据值	操作
☐	1001	phs	5	2024-07-18	7.88052	编辑 删除
☐	1002	phs	10	2024-07-18	7.16944	编辑 删除
☐	1003	phs	15	2024-07-18	6.73289	编辑 删除
☐	1004	phs	20	2024-07-18	8.04385	编辑 删除
☐	1005	phs	25	2024-07-18	7.32814	编辑 删除
☐	1006	phs	30	2024-07-18	8.31519	编辑 删除
☐	1007	phs	35	2024-07-18	7.20741	编辑 删除
☐	1008	phs	40	2024-07-18	7.67923	编辑 删除
☐	2001	phs	45	2024-07-18	8.0809	编辑 删除
☐	2002	phs	50	2024-07-18	8.01743	编辑 删除

显示第 1 到第 10 条记录, 总共 23 条记录 每页显示 10 条记录

© 2024 RuoYi Copyright

图3.5 数据管理

数据监测

- 报警

- 流量报警：流量监测值不在设定范围即发出报警 (100–3000) m³/H
- 压力报警：压力监测值不在设定范围即发出报警 (0.2–0.6) MPA
- 水质报警：余氯监测数值超出阈值范围报警 (0.3–0.5) mg/L
- 浊度监测数值超出阈值范围报警 (1–5) NTU
- pH监测数值超出阈值范围报警 (6.5–8.5)

3. 开发流程

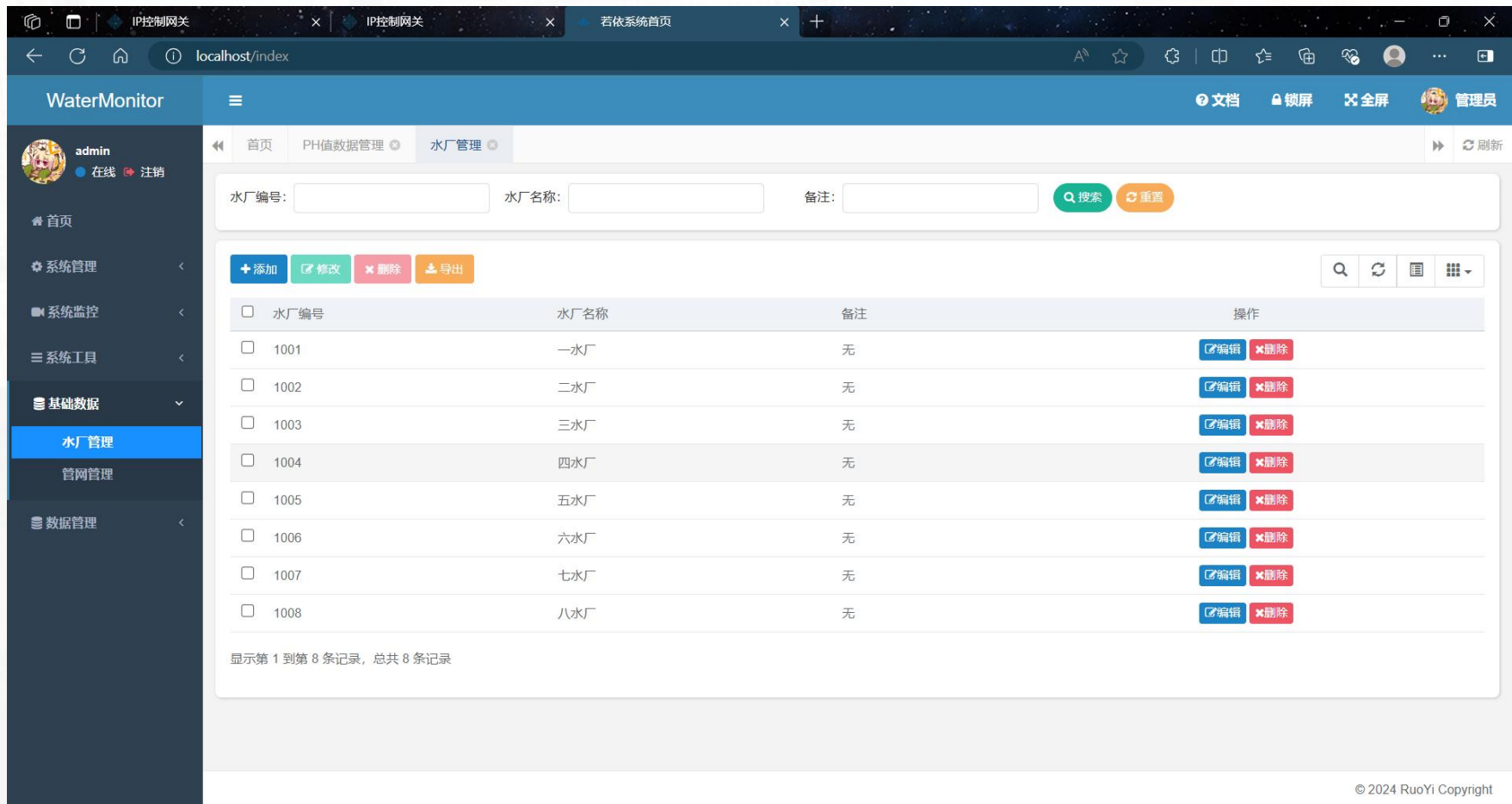


图3.6 数据管理

数据监测

- 水厂

- 出水流量监测：实时监测水厂与泵站出口处流量数据
- 出水压力监测：实时监测水厂与泵站出口处压力数据
- 出水水质监测：实时监测水厂与泵站出口处水质数据

- 管网

- 流量监测：实时监测管网中流量数据
- 压力监测：实时监测管网中压力数据
- 水质监测：实时监测管网中的各个监测点的水质情况

3. 开发流程

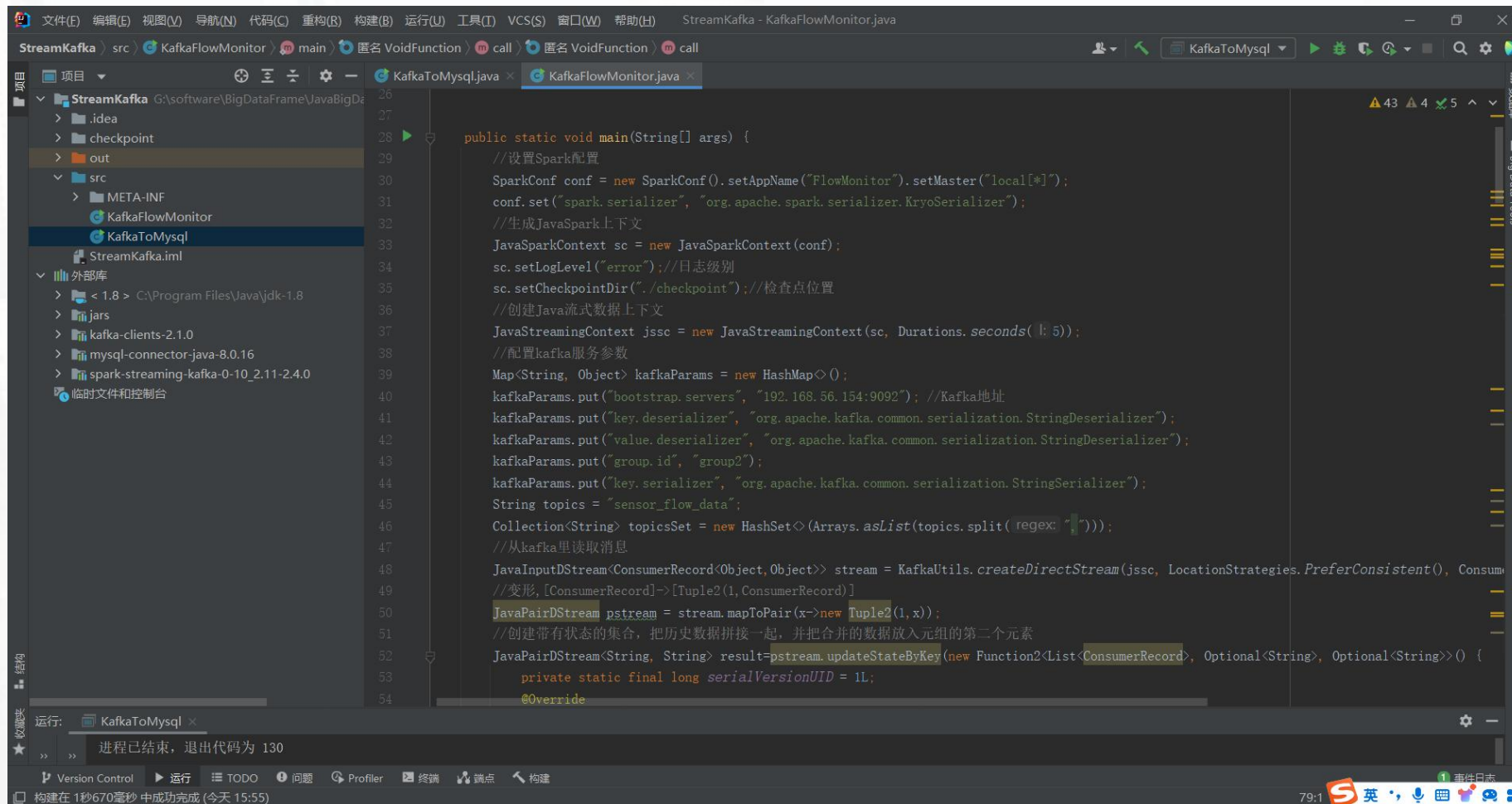


图3.7 Spark服务器从Kafka中获取数据, 将处理结果交给前端数据库

Kafka

Producer：消息生产者，负责将数据发布到Kafka主题（topic）。

Consumer：消息消费者，负责从Kafka主题中读取数据。

Broker：Kafka服务器，负责存储和传递消息。

Topic：消息的逻辑分区，Producer将消息发布到Topic，Consumer从Topic中消费消息。

spark

Spark Core: Spark的基础引擎，负责基本的任务调度、内存管理、故障恢复和存储系统交互。

Spark SQL: 用于结构化数据处理的组件，支持SQL查询。

Spark Streaming: 用于处理实时数据流的组件。

MLlib: 机器学习库，提供常用的机器学习算法。

3. 开发流程

The screenshot shows the Navicat Premium interface with a MySQL database named 'ry' selected. The left sidebar shows the database structure, and the main window displays a list of tables with their properties.

名称	自动递增	修改日期	数据长度	引擎	行	注释
gen_table	3	2024-07-15 20:07:58	16 KB	InnoDB	1	代码生成业务表
gen_table_column	13	2024-07-15 20:07:58	16 KB	InnoDB	6	代码生成业务表字段
sys_config	100	2024-07-15 19:36:35	16 KB	InnoDB	11	参数配置表
sys_dept	200	2024-07-15 19:36:32	16 KB	InnoDB	10	部门表
sys_dict_data	100	2024-07-15 19:36:35	16 KB	InnoDB	29	字典数据表
sys_dict_type	100	2024-07-15 19:36:35	16 KB	InnoDB	10	字典类型表
sys_job	100	2024-07-15 19:36:35	16 KB	InnoDB	3	定时任务调度表
sys_job_log	0		16 KB	InnoDB	0	定时任务调度日志表
sys_logininfor	179	2024-07-18 23:07:25	16 KB	InnoDB	79	系统访问记录
sys_menu	2026	2024-07-15 20:12:32	16 KB	InnoDB	92	菜单权限表
sys_notice	10	2024-07-15 19:36:35	16 KB	InnoDB	3	通知公告表
sys_oper_log	137	2024-07-15 20:15:20	64 KB	InnoDB	37	操作日志记录
sys_post	5	2024-07-15 19:36:32	16 KB	InnoDB	4	岗位信息表
sys_role	100	2024-07-15 19:36:32	16 KB	InnoDB	2	角色信息表
sys_role_dept	0	2024-07-15 19:36:34	16 KB	InnoDB	3	角色和部门关联表
sys_role_menu	0	2024-07-15 19:36:34	16 KB	InnoDB	85	角色和菜单关联表
sys_user	100	2024-07-18 23:07:25	16 KB	InnoDB	2	用户信息表
sys_user_online	0	2024-07-18 23:13:26	16 KB	InnoDB	2	在线用户记录
sys_user_post	0	2024-07-15 19:36:34	16 KB	InnoDB	2	用户与岗位关联表
sys_user_role	0	2024-07-15 19:36:33	16 KB	InnoDB	2	用户和角色关联表
water_flow_count	0	2024-07-18 23:13:45	16 KB	InnoDB	8	
water_flow_trend	0	2024-07-17 22:59:30	16 KB	InnoDB	12	
water_pipes	6	2024-07-18 14:49:25	16 KB	InnoDB	5	
water_pressure_sensor_data	2	2024-07-15 20:31:32	16 KB	InnoDB	1	
water_service_data_monitor	0	2024-07-19 03:44:41	16 KB	InnoDB	11	
water_service_datacount	0	2024-07-18 15:56:05	16 KB	InnoDB	1	
water_service_flow_monitor	0	2024-07-18 15:50:00	64 KB	InnoDB	617	
water_works	9	2024-07-18 14:49:02	16 KB	InnoDB	8	

图3.8 前端数据库

3. 开发流程

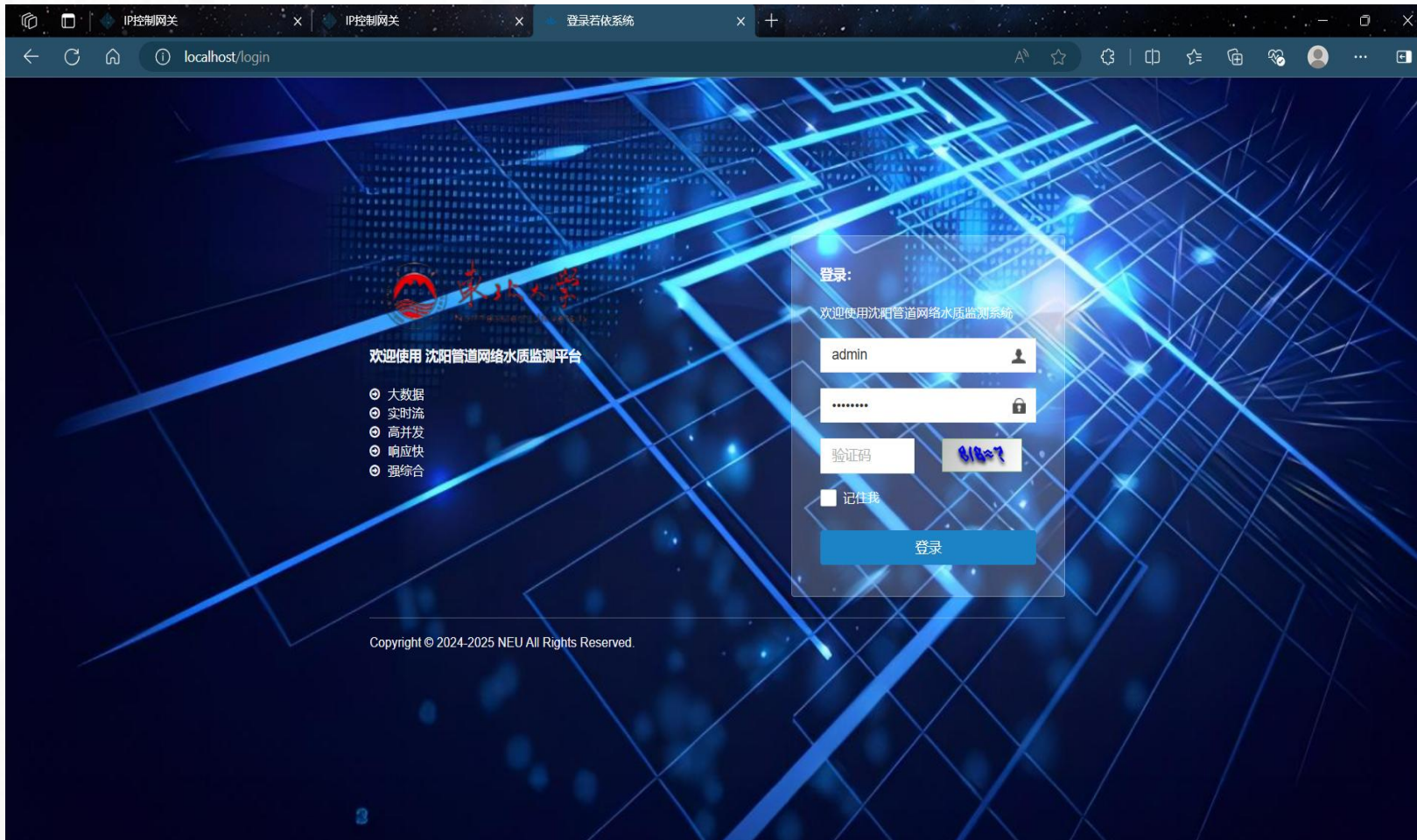


图3.9 前端利用中转数据库的内容展示结果

登录框标题：“登录：”以及欢迎使用沈阳管理高网络水质监测系统的提示。

输入框：

（1）用户名输入框，默认值是“admin”，有一个用户图标。

（2）密码输入框，有一个锁图标。

（3）验证码输入框，旁边为验证码图像。

选项：一个记住我（记住密码）的复选框。

登录按钮：一个蓝色的“登录”按钮。

3. 开发流程



登录成功后修改路径为
`localhost/view/index`
进入系统主界面

图3.10 前端利用中转数据库的内容展示结果

3. 开发流程

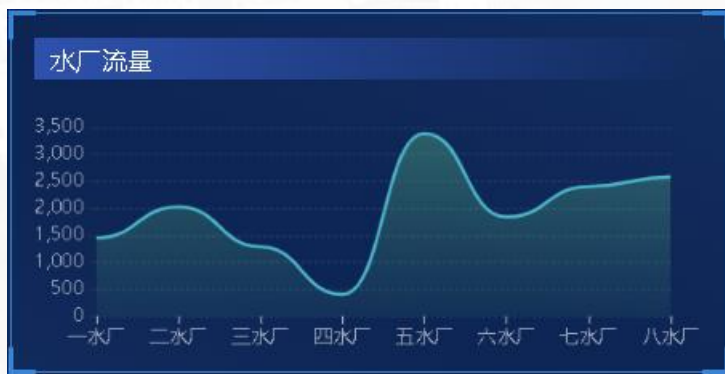


图3.11 水厂流量数据展示



图3.12 指标异常警报

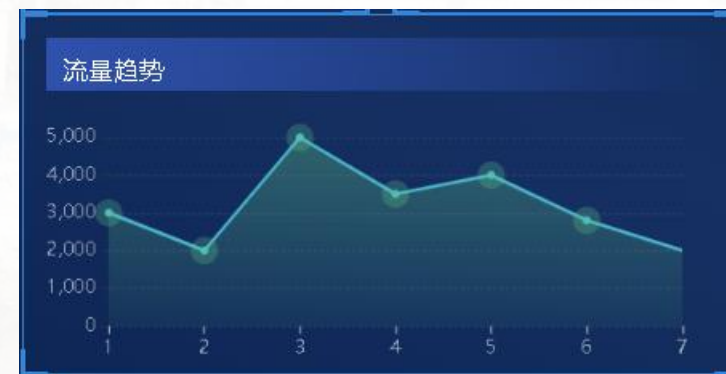


图3.13 流量趋势数据展示



图3.14 系统运行期间所有数据的异常情况统计

图3.15 水厂、水管的各项检测指标，异常数据标红处理

数据监控					
监控地点	时间	水压(MPA)	余氯(mg/L)	浊度(NTU)	酸碱度(PH)
一水厂	2024-07-19 03:43:21	0.25461	0.28708	3.74115	8.45447
二水厂	2024-07-19 03:44:28	0.22221	0.30913	2.6428	7.10464
三水厂	2024-07-19 03:40:51	0.33916	0.40098	4.71842	6.69274
四水厂	2024-07-19 03:43:14	0.59831	0.33738	0.54429	8.00321
五水厂	2024-07-19 03:44:36	0.6081	0.30342	2.52445	7.10438
六水厂	2024-07-19 03:44:08	0.32521	0.3455	5.07953	7.6963

3. 开发流程

流量监控		
监测地点	流量(m3/h)	时间
1008	2583.40916	2024-07-19 03:44:35
1007	2404.41114	2024-07-19 03:44:15
1003	1286.95995	2024-07-19 03:43:55
1004	399.33545	2024-07-19 03:43:35
1008	3329.19077	2024-07-19 03:43:15
1008	555.55306	2024-07-19 03:42:55
1008	2143.00727	2024-07-19 03:42:35
1008	1425.65704	2024-07-19 03:42:15
1005	3392.15145	2024-07-19 03:41:55
2002	2391.95308	2024-07-19 03:41:35

表格的标题为“流量监控”，表头有三列，分别是“监测地点”、“流量(m3/h)”和“时间”。

图3.16 水厂传感器的流量监测数据表

MySQL数据库表设计

水厂

- 基本信息表：水厂名称、水厂编号、备注

对象		water_pipes @ry (root) - 表		water_works @ry (root) - 表	
开始事务	文本	筛选	排序	列	导入 导出 数据生成 创建图表
id	work_id	name	remark		
1	1001	一水厂	无		
2	1002	二水厂	无		
3	1003	三水厂	无		
4	1004	四水厂	无		
5	1005	五水厂	无		
6	1006	六水厂	无		
7	1007	七水厂	无		
8	1008	八水厂	无		

MySQL数据库表设计

管网

- 基本信息表：管网名称、管网编号、备注

对象		water_pipes @ry (root) - 表		water_works @ry (root) - 表	
开始事务		文本 · 筛选 ↓ 排序 列		导入 导出 数据生成 创建图表	
id		pipe_id	name	remark	
▶ 1		2001	青年大街管	无	
2		2002	黄河大街管	无	
3		2003	沈营路管线	无	
4		2004	建设大路管	无	
5		2005	滨河路管线	无	

相关传感器与查询处理数据库的模块

- © ChlorineSensor
- © FlowSensor
- © InitVar
- © PHSensor
- © PressureSensor
- © TurbiditySensor

相关传感器的模块简介

- FlowSensor 类模拟了一个流量传感器的数据生成与发送功能。
- ChlorineSensor 类模拟了一个余氯传感器的数据生成与发送功能。
- PHSensor 类模拟了一个酸碱度传感器的数据生成与发送功能。
- PressureSensor 类模拟了一个压力检测传感器的数据生成与发送功能。
- TurbiditySensor 类模拟了一个浊度传感器的数据生成与发送功能。
- 传感器类操作简介
- 该类创建一个 DatagramSocket 对象，用于将生成的流量传感器数据通过 UDP 协议发送到指定的 IP 地址和端口。
- 传感器数据包括水厂编号、设备类型、设备编号、时间戳和流速。
- 该程序会不断地生成随机的传感器数据，并将其发送到预定的服务器地址和端口。

查询处理数据库的模块简介

- InitVar 类负责初始化项目所需的全局变量。
- * 具体来说，该类用于从 MySQL 数据库中查询水厂和管网的信息，并将这些信息存储到一个全局的 `locationList` 列表中。
- * 这些数据可以被其他类或程序部分用来进行进一步的处理或模拟操作。

4 *Part Four*

总结



智慧水务系统的发展趋势

智能化：通过物联网、大数据等技术实现水务管理的智能化，提高管理效率和水资源利用效率。

信息化：建立信息化平台，实现水务信息的实时监测、分析和共享，提高决策的科学性和准确性。

绿色化：注重环保和可持续发展，推广低能耗、低排放、低污染的水务处理技术，降低对环境的影响。

多元化：满足不同用户的需求，提供多样化的水务服务，如自来水供应、污水处理、雨水排放等。

未来展望

技术创新：随着物联网、大数据等技术的不断发展，智慧水务系统的功能将更加完善，实现更加精准的监测和智能化管理。

智能化管理：智慧水务系统将实现更加智能化的管理，通过数据分析和预测，提高水资源利用效率，降低运营成本。

可持续性发展：智慧水务系统将更加注重环保和可持续发展，通过技术创新和智能化管理，实现水资源的可持续利用。

跨领域合作：智慧水务系统将加强与其他领域的合作，实现跨领域的资源共享和技术创新，共同推动智慧城市的建设和发展。

4. 总结



技术栈丰富

项目的技术栈全面，组员较充分地了解了利用大数据技术开发实际项目的流程，收获颇丰



大数据集分析

项目实现了在数据量较大的情况下利用Spark进行分析统计



实时计算

项目实现了在高并发、多进程环境下的大数据实时计算、实时展示



实际业务

项目聚焦于智能水务系统，面向信息化潮流，处理实际工业中的业务，组员初步接触到了实际的生产环境、工作流程

感谢聆听！

汇报人：XX

2024 年 7 月 19 日